

Relatório Técnico: Plataforma de Agentes Conversacionais com Infraestrutura de Microsserviços

Autores: Bryan Coelho, Diego Mello, Gabriel Gautério Leonardo Pasa e Manuel Soares Da Luz

Disciplina: Engenharia de Software II

Instituto: Pontifícia Universidade Católica de Rio Grande do Sul

1. Introdução

Este relatório documenta as decisões, os experimentos e a implementação do Produto Mínimo Viável (MVP) de uma plataforma extensionista para execução de agentes de Inteligência Artificial (IA) conversacionais. O sistema foi projetado para operar em um ciclo contínuo de raciocínio, ação e observação, utilizando ferramentas externas e modelos de linguagem para resolver requisições dos usuários.

2. Arquitetura da Plataforma

A arquitetura foi estruturada em microsserviços independentes para garantir a escalabilidade, o desenvolvimento desacoplado e a observabilidade. A comunicação síncrona ocorre via protocolo REST, enquanto os eventos assíncronos trafegam por meio do RabbitMQ
[INSERIR DIAGRAMA DE ARQUITETURA AQUI - Conforme requisito do trabalho]

2.1. Componentes e Responsabilidades

Microsserviço	Responsabilidade	Tecnologia Adotada
Agent Service	Recebe requisições, orquestra chamadas ao LLM e ferramentas, mantendo o ciclo agêntico.	Spring Boot (Java) com OpenFeign para comunicação.
LLM Gateway	Proxy unificado para os modelos locais, abstraindo o provedor de IA,	Python (FastAPI) + LiteLLM + Ollama (Modelo Qwen3.5).
API Gateway	Roteamento, rate limiting e ponto de entrada único do sistema.	[Preencher: Spring Cloud Gateway]
Name Server	Descoberta de serviços (Service Registry).	Eureka Server
Memory Service	Armazena o histórico de conversação de curto e longo prazo.	PostgreSQL
Retrieval Service	Busca semântica em documentos (RAG).	[Preencher: Python + ChromaDB ou Qdrant]
Tool Registry	Registra e expõe as ferramentas para os agentes.	Spring Boot

3. Decisões de Arquitetura e Justificativas

Nesta seção, detalhamos as escolhas técnicas, a análise de trade-offs e as dificuldades superadas ao longo do desenvolvimento.

- **LLM Gateway:** Optou-se pela utilização do framework FastAPI com a biblioteca LiteLLM para criar um proxy unificado e resiliente na comunicação com o Ollama local. A escolha do modelo Qwen3.5 (via Ollama) foi baseada na sua eficiência técnica para execução local. As ferramentas (tools) foram mapeadas no DTO de requisição e repassadas de forma estruturada para habilitar o Function Calling.
- **Agent Service:** A implementação foi projetada para focar exclusivamente na orquestração. O serviço não acessa o LLM ou o banco de dados de forma direta; em vez disso, atua orquestrando requisições para os outros serviços. A comunicação interna ocorre via clientes OpenFeign conectados ao Eureka.
- **[Preencher: Decisão sobre o Banco de Dados/Memória]:** Utilizamos o PostgreSQL como banco de dados com a linguagem em python, pois é mais fácil para adicionar os dados no banco, e fácil de visualizar os dados inseridos.
- **[Preencher: Integração do RAG]:** [Adicionar a justificativa do colega responsável]

4. Dependências e Bibliotecas (POM.xml)

Para viabilizar a arquitetura e a integração com os modelos de IA, o projeto faz uso de um conjunto específico de bibliotecas e iniciadores (starters). Abaixo estão listadas as principais dependências configuradas no arquivo de build:

Dependência	Descrição e Propósito
spring-boot-starter-session-data-redis	Gerenciador de sessões HTTP. Centraliza os dados da sessão (logins, estado) no Redis, garantindo resiliência em arquiteturas distribuídas caso um servidor caia.
spring-ai-advisors-vector-store	Gerencia o contexto ("Advisors") injetando informações recuperadas do banco vetorial nos prompts enviados ao LLM, automatizando o pipeline RAG.

Dependência	Descrição e Propósito
spring-ai-starter-model-ollama	Integração direta do Spring com modelos de IA de código aberto (ex: Llama, Qwen) executados localmente via Ollama.
spring-ai-starter-vector-store-qdrant	Conector para o banco vetorial Qdrant, responsável por armazenar e buscar embeddings essenciais para buscas semânticas.
spring-cloud-starter-netflix-eureka-server	Servidor de Descoberta de Serviços (Service Discovery). Atua como catálogo centralizado para localização dinâmica dos microsserviços sem endereços físicos fixados.
postgresql	Driver JDBC em escopo de execução (runtime) para conexão e consultas no banco de dados relacional PostgreSQL.

4.1. Testes e Gerenciamento de Versões

- **Ambiente de Testes:** O componente spring-boot-starter-session-data-redis-test fornece utilitários que provêm um ambiente controlado para testar o comportamento das sessões sem afetar a base do Redis de produção.
- **Gerenciamento (BOMs):** As dependências spring-cloud-dependencies e spring-ai-bom centralizam as definições de versão. Isso assegura compatibilidade estrita e ausência de conflitos internos entre os módulos Cloud e AI.

5. Resiliência e Comunicação Assíncrona

Para evitar falhas em cascata que pudessem comprometer a plataforma, configuramos a resiliência utilizando o padrão Circuit Breaker. Quando o LLM Gateway se encontra

indisponível, o Agent Service interrompe as tentativas de conexão e responde instantaneamente com uma mensagem de fallback.

A mensageria assíncrona foi estruturada utilizando RabbitMQ como broker central. O Agent Service atua como produtor, publicando eventos contínuos de telemetria (como a duração da chamada ao LLM e os recursos acionados) em uma fila específica, sem causar bloqueios (non-blocking) na resposta ao cliente. [Preencher: Outros fluxos assíncronos aplicados pela equipe, como ingestão de PDFs].

6. Observabilidade e Containerização

Todos os microsserviços foram empacotados individualmente através de arquivos Dockerfile e a orquestração do ambiente de desenvolvimento foi feita via docker-compose.yaml. Isso garantiu que a execução local eliminasse totalmente os custos e dependências da computação em nuvem.

[Preencher: Detalhar ferramentas de observabilidade implementadas, como OpenTelemetry, Jaeger para rastreamento distribuído ou Prometheus/Grafana para métricas, além dos insights obtidos].

7. Migração para Produção em Nuvem (Kubernetes)

Para transpor a plataforma do Docker Compose local para um cluster Kubernetes em produção, mapeamos as seguintes alterações arquiteturais necessárias:

- **Descoberta de Serviços:** O servidor Eureka seria desativado em favor do DNS interno nativo do Kubernetes (ex: CoreDNS).
- **Configuração e Segurança:** Variáveis de ambiente codificadas e expostas migrariam para o gerenciamento nativo via ConfigMaps e Secrets.
- **Armazenamento Persistente:** [Preencher: Detalhar a transição de volumes locais do Docker para PersistentVolumeClaims (PVCs) na nuvem]
- **Escalonamento (HPA):** [Preencher: Como o LLM Gateway e o Agent Service seriam escalados automaticamente com base em consumo de CPU]

8. Oportunidades de Melhoria e Riscos Identificados

Avaliando o MVP de forma crítica, destacamos os seguintes riscos e melhorias estruturais:

- **Performance:** Constatamos que o carregamento do modelo na VRAM gera lentidão acentuada no primeiro processamento (Cold Start).

- **Escalabilidade:** [Preencher: Limitações na capacidade simultânea de responder devido ao processamento pesado do LLM]
- **Segurança:** [Preencher: Pontos como a falta de autenticação no API Gateway e a exposição de dados na mensageria]

9. Links de Vídeo:

10. Conclusão

[Preencher: Sintetizar os aprendizados do grupo, a eficácia do padrão de microsserviços e as considerações finais sobre a entrega do projeto].

Com a estrutura de microsserviços, vimos uma praticidade com o desenvolvimento, com cada membro fazendo uma parte dos serviços